Step 2 — Customer Lookups That Run at the Same Time — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: a good billing or technical reply needs the customer's account and recent orders. Those are two separate lookups, and neither needs the other's result — so there's no reason to wait for one before starting the other. This step adds both lookups, first one after another, then in parallel, and measures the difference. **If not:** every billing ticket would wait twice as long as it needs to, for no benefit.

After this step the project runs: billing and technical replies use the account and order details, and `python scripts/time_lookups.py` shows about 2 s one after another vs. about 1 s in parallel. Files not shown here don't change.

Changed first

Changed from Step 1: the billing and technical prompts now have `{account}` and `{orders}` slots. `general.txt` doesn't change — a general question needs no account data.

`src/langchainpro/prompts/billing.txt`

```
You are a billing support agent at SupportCo. Write a short, polite
reply to the customer, under 120 words. Use the account and order
details below. Never promise a refund: say the billing team will
check the charge within 2 business days.

Account: {account}
Recent orders: {orders}

Ticket: {ticket}
```

`src/langchainpro/prompts/technical.txt`

```
You are a technical support agent at SupportCo. Write a short, polite
reply to the customer, under 120 words. Suggest one or two simple
things to try, and ask for one missing detail (browser, device or
steps) if you need it. Use the account details below.
```

```
Account: {account}
Recent orders: {orders}

Ticket: {ticket}
```

Back to this step · Hint 1 · Hint 2 · Solution

Solution — both lookups at the same time with `RunnableParallel`

Story — `chains/lookups.py`: the same two lookups, plus `get_ticket_text()` to carry the ticket through. `build_lookup_step()` puts all three in a `RunnableParallel`: every function gets the same input, they run at the same time, and the result is a dict with the same keys (Doc05's LCEL topic: "a dict becomes a step that runs each value in parallel"). **If not:** the wait grows with every lookup you add.

```
# src/langchainpro/chains/lookups.py
import time

from langchain_core.runnables import Runnable, RunnableParallel

# why: stand-ins for a real accounts database and a real billing API —
# the shape of the data is real, the storage is not
FAKE_ACCOUNTS = {
    "C-1001": "Pro plan, customer since 2023-04-12",
    "C-1002": "Basic plan, customer since 2024-11-03",
    "C-1003": "Pro plan, customer since 2022-01-20",
    "C-1004": "Team plan, customer since 2025-06-30",
}
FAKE_ORDERS = {
    "C-1001": "INV-2201 $29 on 2026-09-01, INV-2202 $29 on 2026-09-01",
    "C-1002": "INV-1870 $9 on 2026-09-03",
    "C-1004": "INV-2310 $99 on 2026-09-05",
}
# why: each fake lookup waits this long, like a real network call —
# without it, "parallel" and "one after another" would look the same
LOOKUP_DELAY_SECONDS = 1

def get_ticket_text(inputs: dict) -> str:
    # why: carries the ticket text through the lookup step, so the prompt
    # after it still has {ticket}
    return inputs["ticket"]

def get_account_info(inputs: dict) -> str:
    time.sleep(LOOKUP_DELAY_SECONDS)
    customer_id = inputs["customer_id"]
    if customer_id in FAKE_ACCOUNTS:
        return FAKE_ACCOUNTS[customer_id]
    return "No account found for this customer."
```

```
def get_order_history(inputs: dict) -> str:
    time.sleep(LOOKUP_DELAY_SECONDS)
    customer_id = inputs["customer_id"]
    if customer_id in FAKE_ORDERS:
        return FAKE_ORDERS[customer_id]
    return "No recent orders."

def build_lookup_step() -> Runnable:
    # why: the two lookups don't need each other's result, so there's no
    # reason to wait for one before starting the other
    # how: RunnableParallel gives every function the same input, runs
    # them at the same time, and returns a dict with the same keys
    # (Doc05's LCEL topic: "a dict becomes a step that runs in parallel")
    return RunnableParallel(
        {
            "ticket": get_ticket_text,
            "account": get_account_info,
            "orders": get_order_history,
        }
    )
```

Changed from Step 1: billing and technical chains start with the lookup step. General stays as it was.

Story — `chains/replies.py`: the lookup step's dict (`ticket` , `account` , `orders`) fills the prompt directly — its keys match the prompt's slots on purpose. **If not:** you'd need glue code to rename keys between the two steps.

```
# src/langchainpro/chains/replies.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable

# new in Step 2
from langchainpro.chains.lookups import build_lookup_step
from langchainpro.models.llm import get_model
from langchainpro.utils.prompts import load_prompt

def build_reply_chain(category: str) -> Runnable:
    # why: this is the branch — one reply chain per category, each with
    # its own prompt file (billing.txt, technical.txt, general.txt)
    # how: the category name IS the prompt file's name
    prompt = ChatPromptTemplate.from_template(load_prompt(category))
    chain = prompt | get_model() | StrOutputParser()
```

```
    # new in Step 2
    if category == "general":
        # why: a general question needs no account data — looking it up
        # anyway would only add a second of waiting
        full_chain = chain
    else:
        # how: the lookup step's dict (ticket, account, orders) fills
        # the prompt's {ticket}, {account} and {orders}
        full_chain = build_lookup_step() | chain

    # changed in Step 2: the chosen chain
    return full_chain
```

Story — `scripts/time_lookups.py`: the proof. It times the two lookups one after another, then the parallel step, and prints both numbers. **If not:** "it's faster" would be a claim, not a measurement.

```
# scripts/time_lookups.py
# Times the two customer lookups: one after another, then in parallel.
# No API key needed. Run from the repo root:
#   python scripts/time_lookups.py
import time

from langchainpro.chains.lookups import (
    build_lookup_step,
    get_account_info,
    get_order_history,
)

def main() -> None:
    inputs = {"ticket": "test ticket", "customer_id": "C-1001"}

    # how: one after another — the second lookup waits for the first
    start = time.time()
    get_account_info(inputs)
    get_order_history(inputs)
    one_by_one = time.time() - start

    # how: the same two lookups, started at the same time
    start = time.time()
    build_lookup_step().invoke(inputs)
    in_parallel = time.time() - start

    print(f"one after another: {one_by_one:.1f} s")
    print(f"in parallel:       {in_parallel:.1f} s")
```

```
if __name__ == "__main__":
    main()
```

Story — `tests/unit/test_lookups.py`: known and unknown customers, the three keys, and a time check — all with no API key. **If not:** a later change could quietly make the lookups run one after another again.

```
# tests/unit/test_lookups.py
# Checks the fake lookups and that the lookup step runs them in
# parallel. No API key needed. Run from the repo root:
#   python tests/unit/test_lookups.py
import time

from langchainpro.chains.lookups import (
    build_lookup_step,
    get_account_info,
    get_order_history,
)

def test_known_customer() -> None:
    inputs = {"ticket": "hi", "customer_id": "C-1001"}
    print(f"case 1 (account)          -> {get_account_info(inputs)}")
    print(f"case 1 (orders)           -> {get_order_history(inputs)}")

def test_unknown_customer() -> None:
    inputs = {"ticket": "hi", "customer_id": "C-9999"}
    print(f"case 2 (account)          -> {get_account_info(inputs)}")
    print(f"case 2 (orders)           -> {get_order_history(inputs)}")

def test_lookup_step() -> None:
    inputs = {"ticket": "hi", "customer_id": "C-1002"}
    start = time.time()
    result = build_lookup_step().invoke(inputs)
    seconds = time.time() - start
    keys = sorted(result.keys())
    print(f"case 3 (keys)             -> {keys}")
    # why: two 1-second lookups — about 1 s means they ran in parallel,
    # about 2 s means one after another
    print(f"case 3 (under 1.5 s?)     -> {seconds < 1.5}")

if __name__ == "__main__":
    test_known_customer()
    test_unknown_customer()
    test_lookup_step()
```

Run it:

```
python scripts/time_lookups.py
python tests/unit/test_lookups.py
python -m langchainpro.main
```

`time_lookups.py` prints:

```
one after another: 2.0 s
in parallel:       1.0 s
```

`test_lookups.py` prints:

```
case 1 (account)          -> Pro plan, customer since 2023-04-12
case 1 (orders)           -> INV-2201 $29 on 2026-09-01, INV-2202 $29
on 2026-09-01
case 2 (account)          -> No account found for this customer.
case 2 (orders)           -> No recent orders.
case 3 (keys)             -> ['account', 'orders', 'ticket']
case 3 (under 1.5 s?)     -> True
```

(The second line is shown wrapped onto 2 lines to fit the page — really one line of output.)